

Auditoría funcional y de ciberseguridad

IMDIMAQ — Sistema interno de punto de venta y gestión comercial.

Repositorio `DemoSystem-Front`, rama `main`.

4	7	3	24
MÓDULOS ENTREGADOS	MÓDULOS PENDIENTES	HALLAZGOS ALTOS	CVE EN DEPENDENCIAS

El proyecto compila sin errores en desarrollo y producción. El bundle inicial de producción es de **326 kB** (94 kB transferidos), muy por debajo del presupuesto configurado de 800 kB. No se detectaron secretos, credenciales ni artefactos de build versionados.

FECHA DEL ANÁLISIS

6 de agosto de 2026

STACK

Angular 20.3.26 · TypeScript 5.8 · pnpm 10.13

COMMIT AUDITADO

9c01a8f – Integración Frontend con backend fase 3

ALCANCE

Análisis estático del repositorio frontend

PARTE I

Auditoría funcional

Contraste entre lo propuesto en `docs/architecture.md` y `docs/api-contract.md` y lo efectivamente implementado en el código.

1. Cumplimiento del plan de módulos

#	MÓDULO PROPUESTO	ESTADO	OBSERVACIÓN
1	Base visual, autenticación, layout y dashboard	COMPLETO	Sistema de diseño en OKLCH, guards, shell y métricas reales
2	Catálogo: categorías, unidades y productos	COMPLETO	Incluye especificaciones e imágenes
3	Inventario	COMPLETO	Movimientos, stock bajo y consulta de stock
4	Clientes	PLACEHOLDER	Ruta y menú reservados
5	Cotizaciones	PLACEHOLDER	Ruta y menú reservados
6	Ventas / POS	PLACEHOLDER	Ruta y menú reservados
7	Pagos y cuentas por cobrar	PLACEHOLDER	Ruta y menú reservados
8	Contabilidad básica	AUSENTE	Sin ruta ni ítem de menú
9	Reportes	PLACEHOLDER	Ruta y menú reservados
10	Configuración, usuarios y auditoría	PARCIAL	Usuarios completo · Configuración placeholder · Auditoría sin ruta

Los módulos 1 a 3 más la gestión de usuarios están terminados y consumen el backend REST real. **Contabilidad y auditoría son los únicos ítems del plan sin lugar reservado** en `app.routes.ts` ni en la navegación del shell.

2. Convenciones de arquitectura

Se cumplen sin excepciones relevantes: componentes standalone, carga diferida por feature, `ChangeDetectionStrategy.OnPush` en todos los componentes, estado con signals y transporte HTTP centralizado en `core/http` mediante tres interceptores encadenados.

La excepción es `src/app/shared/` : está vacío (solo contiene un README) mientras existe duplicación real entre features: `errorMessage()` copiado en 4 archivos, `toQueryParams()` en 2, y la paginación, los badges de estado y los modales reimplementados en cada página.

Conviene extraer `shared/` **antes** de arrancar Clientes, Cotizaciones y Ventas, o la duplicación se multiplicará por siete.

3. Desviación respecto del contrato documentado

El documento `docs/api-contract.md` define la respuesta de error como `{ errorCode, message, details, traceId }`, pero `api-error.interceptor.ts` parsea el formato real de NestJS (`statusCode`, `message`, `error`, `path`) y sintetiza un código artificial cuando falta:

```
typeof payload.error === 'string'
  ? payload.error
  : `HTTP_${error.status} || 'NETWORK'`
```

Consecuencias:

- `errorCode` no es el código estable que promete el documento, por lo que no sirve para decidir comportamiento en el frontend.
- `traceId` está declarado en `ApiRequestError` pero **nunca se llena** → no hay trazabilidad hacia el backend.
- `core/models/api-error.model.ts` es código muerto: ningún archivo lo importa.

La paginación (`data / page / limit / total / totalPages`) y la autenticación por cookie `HttpOnly` sí coinciden con lo documentado.

4. Defectos funcionales

F-1 `imageUrl()` lanza una excepción en producción

`frontend/src/app/features/catalog/catalog.service.ts:129`

```
new URL(fileUrl, this.config.baseUrl)
```

En producción `apiBaseUrl` es `/api/v1` (relativo) y `new URL(x, '/api/v1')` lanza `Invalid URL`, verificado en Node. **Las imágenes de producto no renderizan en el entorno desplegado**, tanto en la grilla como en el detalle. En desarrollo, con un `fileUrl` relativo, además pierde el segmento `/v1`.

CORRECCIÓN

Resolver contra `document.baseURI` o `window.location.origin` cuando `baseUrl` no sea absoluto, o devolver la ruta tal cual y dejar que el navegador la resuelva.

F-2 El rol por defecto es ADMIN

`frontend/src/app/core/auth/auth.service.ts:50`

```
readonly role = computed(() => this.session()?.role ?? 'ADMIN');
```

Sin sesión activa devuelve administrador. Hoy queda tapado porque `authGuard` se ejecuta antes que `roleGuard`, pero es un default que **falla en abierto**: cualquier consumo fuera de rutas protegidas, o un cambio en el orden de los guards, expone el menú y las acciones de administración.

CORRECCIÓN

Devolver `null` y que `roleGuard`, `canManage()` y `canWrite()` denieguen ante ausencia de rol.

F-3 Un backend caído expulsa al usuario al login

frontend/src/app/core/auth/auth.service.ts:74-84

`ensureSession()` marca `initialized = true` en el `finalize` incluso cuando el error es un 500 o una caída de red; el error se propaga al `guard`, la navegación falla y no hay reintento ni mensaje al usuario. Solo el 401 está tratado explícitamente.

CORRECCIÓN

Distinguir «no autenticado» (401) de «backend no disponible»: en el segundo caso no marcar la sesión como inicializada y mostrar una pantalla de reintento.

F-4 El comando de pruebas no existe y no hay ni una sola prueba

frontend/package.json · frontend/angular.json

`package.json` declara `"test": "ng test --watch=false"`, pero `angular.json` no tiene `target test`, no hay `karma` ni `jasmine` en las dependencias y existen **0 archivos** `.spec.ts`. `tsconfig.spec.json` quedó huérfano.

Queda sin cobertura lógica que sí la merece: los cuatro `guards`, el interceptor de errores y los validadores `validateProduct`, `validateMovement` y `validateForm`.

F-5 LoadingService no se consume en ninguna parte

frontend/src/app/core/services/loading.service.ts

El `loadingInterceptor` cuenta peticiones en vuelo, pero `isLoading` no aparece en ninguna plantilla; cada página reimplementa sus propios flags de carga. O se conecta a una barra de progreso global, o el servicio y su interceptor sobran.

F-6 confirm() nativo en catálogo frente a modal propio en usuarios

catalog.page.ts:291, 397, 504, 566, 605

Cinco confirmaciones del módulo de catálogo usan el diálogo nativo del navegador, mientras usuarios implementa un modal propio coherente con el sistema de diseño. Es inconsistente, bloquea el hilo y no es testeable.

5. Accesibilidad e internacionalización

- **Modales sin focus trap ni cierre con Escape.** El marcado es correcto (`role="dialog"`, `aria-modal`, `aria-labelledby`), pero el foco no entra al abrir ni vuelve al disparador al cerrar.
- **Avisos de éxito sin `aria-live`.** Los errores usan `role="alert"` correctamente; los `notice()` de éxito no se anuncian a un lector de pantalla.
- **Sin `LOCALE_ID` es-PE registrado.** `DatePipe` formatea en inglés donde no se pasa formato explícito, y `money()` es un `toFixed(2)` sin símbolo de moneda ni separador de miles.

Auditoría de ciberseguridad

Revisión estática del código, las dependencias y la configuración de despliegue del frontend. La severidad se asigna considerando que el backend es la autoridad real de seguridad.

6. Alcance y limitaciones

Esta auditoría cubre exclusivamente el repositorio frontend. **No incluye:**

- El backend NestJS, donde residen la autenticación, la autorización efectiva y las reglas de negocio.
- Pruebas dinámicas (DAST) ni pruebas de penetración sobre un entorno desplegado.
- La configuración real de la cookie de sesión (`SameSite` , `Secure` , `HttpOnly` , expiración), que fija el servidor y no es observable desde este repositorio.

Las conclusiones que dependen del backend se señalan explícitamente como puntos a confirmar con ese equipo.

7. Controles verificados como correctos

- **Cero almacenamiento de credenciales en el cliente.** Sin coincidencias de `localStorage` , `sessionStorage` , `document.cookie` ni `indexedDB` en todo `src/app` . La sesión vive únicamente en la cookie `HttpOnly` , que JavaScript no puede leer.
- **Sin fugas por consola.** Cero llamadas a `console.*` en el código de aplicación.
- **Sin secretos en el repositorio.** Ningún `.env` , clave ni credencial versionada; `dist/` y `.angular/` correctamente ignorados (0 archivos rastreados de ambos).
- **Sin source maps en producción.** `sourceMap` solo se activa en la configuración `development` .
- **Sin redirección abierta.** Todas las llamadas a `navigateByUrl` usan literales fijos; no se lee ningún `returnUrl` de la query string.
- **Escapado por defecto.** `strict: true` y `strictTemplates` activos; toda la salida de datos del backend pasa por interpolación de Angular, que escapa HTML.
- **Campos de contraseña correctos.** Los cinco usan `type="password"` con el `autocomplete` apropiado (`current-password` / `new-password`) y nunca viajan por GET.

8. Hallazgos de seguridad

ID	SEVERIDAD	HALLAZGO	RESPONSABLE
S-1	ALTA	Angular 20.3.26 con tres CVE altas sin parchear	Frontend
S-2	ALTA	Sin protección CSRF con autenticación por cookie	Frontend + Backend
S-3	ALTA	Autorización con default permisivo (falla en abierto)	Frontend
S-4	MEDIA	Sin Content-Security-Policy ni cabeceras de seguridad	Infraestructura
S-5	MEDIA	Recursos de terceros sin integridad (SRI)	Frontend
S-6	MEDIA	bypassSecurityTrustHtml instalado en el shell	Frontend
S-7	MEDIA	Mensajes de error del backend expuestos sin filtrar	Frontend
S-8	MEDIA	Contraseña temporal persistente en el portapapeles	Frontend
S-9	MEDIA	Sin cierre de sesión por inactividad	Frontend
S-10	MEDIA	Validación de subida de archivos solo en cliente	Backend
S-11	MEDIA	21 CVE adicionales en dependencias de build	Frontend
S-12	BAJA	Política de contraseñas divergente cliente / servidor	Backend
S-13	BAJA	Sin manejo de 429 en el login	Frontend
S-14	BAJA	Comparación de origen por <code>startsWith</code> en el interceptor	Frontend
S-15	BAJA	Sin trazabilidad ni módulo de auditoría	Frontend + Backend

9. Detalle de hallazgos altos

S-1 Angular 20.3.26 con tres CVE altas sin parchear

frontend/package.json – versión instalada 20.3.26

PAQUETE	VULNERABILIDAD	AVISO
@angular/core	XSS vía atributos manejadores de evento en i18n	GHSA-jj27-h5hq-8x99
@angular/compiler	XSS vía atributos manejadores de evento en i18n	GHSA-jj27-h5hq-8x99
@angular/common	Ambigüedad de clave en <code>HttpTransferCache</code> → reutilización de respuestas entre peticiones distintas y envenenamiento de estado	GHSA-jhpw-976m-542j

Las tres están corregidas en **20.3.27**. El rango declarado `^20.3.0` ya admite el parche: es una actualización de un solo comando, sin cambios de código.

Explotabilidad real hoy: baja, porque la aplicación no usa i18n de Angular ni SSR con `HttpTransferCache`. Pero deja de serlo en cuanto se active cualquiera de las dos, y el coste de parchear es nulo.

CORRECCIÓN

```
pnpm update @angular/core @angular/common @angular/compiler @angular/forms @angular/router  
@angular/platform-browser @angular/animations
```

S-2 Sin protección CSRF con autenticación por cookie

frontend/src/app/core/http/credentials.interceptor.ts · app.config.ts

La sesión se sostiene en una cookie `HttpOnly` y **todas** las peticiones salen con `withCredentials: true`. El navegador adjunta esa cookie automáticamente también cuando la petición la origina otro sitio.

El frontend **no envía ningún token anti-CSRF ni cabecera personalizada**, y `provideHttpClient()` se configura sin `withXsrfConfiguration()`. Si el backend no fija `SameSite=Strict` o `Lax` en la cookie de sesión, un sitio malicioso visitado por un usuario autenticado puede provocar operaciones de escritura a su nombre:

- `POST /users`, crear un usuario administrador.
- `POST /users/:id/reset-password`, invalidar credenciales ajenas.
- `POST /products/:id/deactivate`, `POST /inventory/exits`, alterar catálogo e inventario.

CORRECCIÓN

Confirmar con el equipo de backend el atributo `SameSite` de la cookie. Si no es `Strict` / `Lax`, implementar doble envío de token: el backend emite una cookie legible `XSRF-TOKEN` y el frontend la reenvía con `withXsrfConfiguration({ cookieName, headerName })`.

ES LA DECISIÓN DE SEGURIDAD PENDIENTE MÁS IMPORTANTE DEL PROYECTO.

S-3 Autorización con default permisivo

`auth.service.ts:50 · role.guard.ts · catalog.page.ts:72 · inventory.page.ts:47`

Es el mismo defecto que **F-2**, visto desde la seguridad. `role()` devuelve `'ADMIN'` cuando no hay sesión: el estado de «desconocido» se resuelve como el rol de máximo privilegio, en lugar de denegar.

El propio código lo documenta correctamente («ocultar rutas no es seguridad»), y esa premisa es la adecuada. El riesgo no es que un atacante evada `roleGuard` (siempre podrá hacerlo desde devtools), sino que **el backend confíe en que el frontend filtró**.

CORRECCIÓN

Dos acciones: (1) cambiar el default a `null` y denegar por ausencia de rol; (2)

CONFIRMAR CON EL EQUIPO DE BACKEND

que cada endpoint de escritura de catálogo, inventario y usuarios valida el rol del lado servidor, y no solo la autenticación.

10. Detalle de hallazgos medios

S-4 Sin Content-Security-Policy ni cabeceras de seguridad

El repositorio no contiene Dockerfile, configuración de nginx ni ningún artefacto de despliegue, por lo que no hay CSP, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` ni `Strict-Transport-Security` definidos en ninguna parte. Sin CSP, cualquier XSS futuro escala a compromiso total de la sesión; sin `X-Frame-Options` / `frame-ancestors`, la aplicación es encuadrable (clickjacking sobre acciones destructivas como bloquear usuarios).

CORRECCIÓN

Añadir al repositorio la configuración del servidor estático con CSP restrictiva y las cabeceras anteriores. Debe incluir además `try_files $uri /index.html` para que las rutas lazy sobrevivan a un refresco.

S-5 Recursos de terceros sin integridad

`frontend/src/index.html`

La hoja de estilos y las fuentes se cargan desde `fonts.googleapis.com` y `fonts.gstatic.com` sin atributo `integrity`. Un compromiso de ese origen permite inyectar CSS arbitrario, con el que se puede exfiltrar el contenido de campos de formulario mediante selectores de atributo y peticiones de fondo.

Para un sistema interno hay además un motivo operativo: la aplicación depende de un CDN externo para renderizar correctamente.

CORRECCIÓN

Auto-hospedar Inter e IBM Plex Mono como assets del proyecto y eliminar los `preconnect`. Resuelve el riesgo, mejora el tiempo de carga y permite una CSP sin `style-src` externo.

S-6 **bypassSecurityTrustHtml instalado en el shell**

frontend/src/app/layout/app-shell/app-shell.component.ts:69

Los iconos de navegación se construyen como cadenas SVG y se inyectan con `[innerHTML]` tras desactivar el saneado de Angular.

Hoy es seguro: el diccionario `ICONS` es una constante del módulo y `item.icon` proviene de un array literal; ningún dato externo alcanza esa ruta. El problema es que el patrón queda disponible y normalizado: el día que los iconos o las etiquetas de menú vengan del backend, se convierte en XSS directo sin que nadie note el cambio.

CORRECCIÓN

Reemplazar por elementos `<svg>` estáticos en la plantilla o un componente de icono con `@switch`, y eliminar la dependencia de `DomSanitizer`.

S-7 **Mensajes de error del backend expuestos sin filtrar**

frontend/src/app/core/http/api-error.interceptor.ts

El interceptor toma el campo `message` del cuerpo de error y lo propaga tal cual hasta la interfaz, para cualquier código de estado. No es XSS, porque Angular escapa la interpolación, pero un 500 de NestJS o de Prisma puede filtrar al usuario final nombres de tabla, nombres de *constraint*, rutas del servidor o fragmentos de consulta.

CORRECCIÓN

Mostrar el `message` del backend solo para códigos 4xx conocidos (400, 409, 422) y sustituirlo por un texto genérico en 5xx, registrando el detalle junto al `traceId`.

S-8 **Contraseña temporal persistente en el portapapeles**

frontend/src/app/features/users/users.page.ts:267 · users.page.html:288

Al restablecer una contraseña, el modal muestra la clave temporal en claro y ofrece copiarla.

`navigator.clipboard.writeText()` la deja en el portapapeles del sistema **indefinidamente**, accesible a cualquier aplicación del equipo y, en portapapeles sincronizados, a otros dispositivos. El modal tampoco tiene temporizador ni auto-cierre, por lo que la clave puede quedar visible en pantalla en un puesto compartido.

CORRECCIÓN

Ocultar la clave tras un temporizador visible (p. ej. 60 s), cerrar el modal al expirar y limpiar el portapapeles al cerrarlo. Idealmente, entregar la clave por un canal fuera de banda en vez de mostrarla.

S-9 **Sin cierre de sesión por inactividad**

La única reacción existente es al 401 del backend. Un terminal de punto de venta desatendido mantiene la sesión abierta hasta que expire la cookie del servidor, y no hay aviso previo de expiración ni bloqueo de pantalla. En un entorno POS, donde el equipo es compartido y está en zona de atención al público, es un riesgo operativo concreto.

CORRECCIÓN

Temporizador de inactividad en el shell (p. ej. 15 min) con aviso a los 14, que invoque `logout()`. Debe acompañarse de una expiración equivalente en el servidor.

S-10 Validación de subida de archivos solo en cliente

frontend/src/app/features/catalog/catalog.page.ts:572-592

`uploadImage()` valida el tipo por `file.type` (que el cliente controla y puede falsificar) y el tamaño máximo de 5 MB en el navegador. Ambas son validaciones de usabilidad, no de seguridad.

CORRECCIÓN (BACKEND)

Revalidar el tipo real por firma de archivo, imponer el límite de tamaño, normalizar el nombre, almacenar fuera de la raíz web y servir las imágenes con `Content-Type` no ejecutable y `X-Content-Type-Options: nosniff`.

S-11 21 CVE adicionales en dependencias de build

`pnpm audit` reporta 24 vulnerabilidades en total (11 altas, 13 moderadas). Descontadas las tres de Angular en tiempo de ejecución (S-1), las 21 restantes están en el árbol de `@angular/cli` y `@angular-devkit/build-angular`: `webpack-dev-server`, `postcss`, `js-yaml`, `brace-expansion`, `tar`, `undici`, `ip-address`, `fast-uri`, `hono` y `uuid`.

No llegan al bundle de producción, por lo que no exponen a los usuarios finales. Sí afectan a la máquina del desarrollador: las dos de `webpack-dev-server` permiten, en escenarios concretos, que un sitio malicioso lea código fuente mientras el servidor de desarrollo está activo.

CORRECCIÓN

Actualizar la cadena de herramientas (`pnpm update @angular/cli @angular-devkit/build-angular`) e incorporar `pnpm audit` a la integración continua, una vez exista.

11. Detalle de hallazgos bajos

S-12 Política de contraseñas divergente cliente / servidor

El frontend exige 12 caracteres como mínimo con al menos una letra y un dígito, en `change-password.page.ts` y en la creación de usuarios. Es una validación de UX: el servidor debe imponer la misma política o una más estricta. Conviene además dejarla documentada en un único lugar para que ambas partes no se separen con el tiempo.

S-13 Sin manejo de 429 en el login

El limitado de intentos corresponde al backend, pero el frontend no contempla el código 429: caería en el mensaje genérico «Ocurrió un error inesperado», que confunde al usuario legítimo bloqueado por intentos fallidos. Añadir un caso explícito en `defaultMessage()`.

S-14 Comparación de origen por startsWith

frontend/src/app/core/http/credentials.interceptor.ts

El interceptor decide si adjuntar credenciales con `request.url.startsWith(config.baseUrl)`. Es correcto hoy, porque en producción `baseUrl` es la ruta relativa `/api/v1`. Pero si en el futuro se apunta a un host absoluto, una URL como `https://api.midominio.com.atacante.io/` superaría la comparación y las cookies de sesión se enviarían a un tercero.

CORRECCIÓN

Comparar por origen resuelto (`new URL(request.url, document.baseURI).origin`) en lugar de por prefijo de cadena.

S-15 Sin trazabilidad ni módulo de auditoría

El módulo 10 del plan incluye auditoría, y no existe ni la ruta. Del lado del transporte, `ApiRequestError` declara un campo `traceId` que el interceptor nunca llena, de modo que un error reportado por un usuario no puede correlacionarse con los registros del servidor. Es el pilar de detección y respuesta que hoy falta por completo.

Plan de acción recomendado

Ordenado por relación entre riesgo mitigado y esfuerzo requerido.

ORD.	ACCIÓN	REFERENCIA	ESFUERZO	MOTIVO
1	Actualizar Angular a 20.3.27	S-1	Minutos	Tres CVE altas, un solo comando, sin cambios de código
2	Corregir <code>imageUrl()</code>	F-1	Minutos	Fallo confirmado en producción
3	Cambiar el rol por defecto a <code>null</code>	F-2 · S-3	Minutos	Elimina un default que falla en abierto
4	Definir la estrategia CSRF con el backend	S-2	Días	Riesgo de escritura no autorizada en toda la API
5	Añadir despliegue con CSP y cabeceras	S-4	Horas	Contiene cualquier XSS futuro; hoy no existe artefacto de despliegue
6	Auto-hospedar las fuentes	S-5	Horas	Elimina dependencia y superficie de terceros
7	Alinear <code>api-contract.md</code> con el interceptor	§3	Horas	El contrato documentado no describe el sistema real
8	Habilitar <code>ng test</code> con pruebas de guards, interceptor y validadores	F-4	Días	Cero cobertura sobre la lógica de seguridad del cliente
9	Extraer <code>shared/</code> antes de los módulos 4 a 7	§2	Días	Evita multiplicar por siete la duplicación existente
10	Timeout de inactividad y endurecer el modal de clave temporal	S-8 · S-9	Horas	Riesgo operativo propio de un terminal POS compartido

Valoración general. La base es sólida y notablemente disciplinada: arquitectura coherente, tipado estricto, transporte HTTP centralizado, cero credenciales en el cliente y cero secretos en el repositorio. Los tres hallazgos altos son de corrección rápida, y dos de ellos (S-1 , S-3) se resuelven en minutos.

Las dos deudas estructurales que conviene saldar antes de continuar con los módulos pendientes son **la ausencia total de pruebas** y el `shared/ vacío`: ambas se vuelven progresivamente más caras con cada feature nueva que se añade.